1. What is the difference between forEach and for...of? When would you use each?

Both forEach and for...of are used to iterate over elements in an array, but they work differently.
forEach is an array method that executes a callback function once for each element in the array. It cannot be stopped early using break or continue.
for...of is a loop that iterates over iterable objects such as arrays, strings, and sets. It allows using break, continue, and return, giving more control over the loop.

When to use each?

Use forEach when you want to perform an operation on every element in an array and you don't need to stop the loop.
Use for...of when you need more control over the iteration, such as using break, continue, or working with asynchronous code using await.

2. What is hoisting and what is the Temporal Dead Zone (TDZ)? Explain with examples.

Hoisting is a JavaScript behavior where variable and function declarations are processed before the code is executed.
When using var, the variable is hoisted and initialized with undefined. This means you can access it before its declaration,but its value will be undefined.
Example:

```
console.log(x);
var x = 10;
```

Output:
Undefined

With let and const, the variables are also hoisted, but they are not initialized immediately. The time between entering the scope and declaring the variable is called the Temporal Dead Zone (TDZ).If you try to access the variable during this period, JavaScript throws a ReferenceError.
Example:

```
console.log(age);
let age = 20;
```
Output:
ReferenceError

* The TDZ helps prevent using variables before they are properly initialized, making the code safer and easier to understand.

3. What are the main differences between == and ===?

The “==” operator compares values after performing **type coercion**, which means JavaScript may automatically convert one data type to another before comparing.

Example:
```
5 == "5"
```
Output:
true
The === operator is called the strict equality operator. It compares both the value and the data type without converting anything.

Example:
```
5 === "5"
```
Output:
false

4. What’s the difference between type conversion and coercion?

Provide examples of each.

Both type conversion and type coercion change the data type of a value, but the difference is who performs the conversion.

* Type conversion (explicit conversion) happens when the programmer converts the data type manually using functions like Number(), String(), or Boolean().
Example:

```
let num = Number("25");
```

Output:
25

* Type coercion (implicit conversion) happens automatically by JavaScript when different data types are used together.

Example:

```
"5" + 2
```

Output:
"52"

Another example:

```
"5" - 2
```

Output:
3

5. Explain how try...catch works and why it is important in async operations.

The try...catch statement is used to handle errors in JavaScript. The code that may cause an error is placed inside the try block. If an error occurs, JavaScript stops executing the code inside the try block and jumps to the catch block, where the error can be handled without crashing the program.

In asynchronous operations such as async/await, try...catch is important because it catches errors that may occur while waiting for a Promise to resolve. This helps prevent the application from stopping unexpectedly and allows you to display an error message or take another appropriate action.